



PRÉ-RENTRÉE 2026-2027

Stage d'entrée en Première NSI

Python : des fondations au mini-projet

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document de présentation - familles et élèves

Un stage Python conçu pour démarrer la spécialité sans lacunes

La NSI n'ayant pas été enseignée auparavant, le stage commence au bon niveau : aucune connaissance préalable de Python n'est exigée. L'objectif est de construire en dix heures un socle opérationnel permettant de suivre la Première NSI avec méthode.

Organisation

- 5 séances de 2 heures ;
- 2 élèves ;
- un ordinateur par élève ;
- alternance théorie courte, programmation, tests et oral ;
- un fil rouge : analyser des données de capteurs ;
- un dossier élève et un bilan final individualisé.

Les cinq séances

Séance	Objectif	Compétences Python	Livrable
1	Comprendre l'exécution	variables, types, affectation, booléens, <code>if</code>	premier programme commenté
2	Automatiser	<code>for</code> , <code>range</code> , <code>while</code> , compteurs, accumulateurs	outils d'analyse d'une liste
3	Structurer et fiabiliser	fonctions, paramètres, <code>return</code> , <code>None</code> , tests	module de fonctions testées
4	Manipuler des données	listes, tuples, dictionnaires, CSV	analyseur de table
5	Résoudre un problème complet	recherche, tri, dichotomie, projet	projet Python documenté

Ce que l'élève saura faire

À la fin du stage, l'élève sera capable de lire et tracer un programme, écrire des conditions et des boucles, structurer un code en fonctions, manipuler des listes et des dictionnaires, importer un CSV, écrire des tests et présenter un mini-projet.

Deux parcours dans le même groupe

- **Parcours Fondations guidées** : pour un élève qui débute Python et n'a pas suivi de NSI auparavant ;
- **Parcours Fiabilisation et autonomie** : pour un candidat individuel qui connaît déjà plusieurs notions, mais doit rendre son code plus fiable, testé et explicable.

Le stage ne cherche pas à “faire toute la Première en avance”. Il rend disponible le langage Python qui sera mobilisé toute l'année dans les données, les algorithmes et les projets.

Les repères annuels couverts

- programmation séquentielle, affectations et expressions ;
- logique booléenne et conditions ;
- boucles, compteurs, accumulateurs et terminaison ;
- fonctions, contrats, documentation et assertions ;
- listes, tuples, dictionnaires et tables CSV ;
- recherche séquentielle, tri par insertion et recherche dichotomique ;
- premières comparaisons de coûts ;
- conduite d'un projet : spécifier, découper, coder, tester et présenter.

Matériel, évaluation et livrables

- un ordinateur par élève, avec Python 3 ou Thonny ;
- un mini-diagnostic pratique au démarrage ;
- un fichier Python conservé après chaque séance ;
- un memento Python de Première ;
- un mini-projet final documenté et testé ;
- un bilan individualisé avec plan de travail pour l'année.



PRÉ-RENTRÉE 2026-2027

Installer l'environnement

Python 3 et Thonny

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Guide famille et élève

Configuration recommandée

- un ordinateur portable par élève ;
- Python 3.11 ou supérieur ;
- Thonny récent ;
- 2 Go d'espace libre ;
- un dossier local de travail sauvegardé ;
- le pack de code élève correspondant, sans solution.

Installation

Windows et macOS

1. Télécharger Thonny depuis son site officiel.
2. Installer avec les options par défaut.
3. Ouvrir Thonny.
4. Vérifier dans **Outils > Options > Interpréteur** que Python 3 est sélectionné.
5. Exécuter :

```
print("Nexus NSI prêt")
```

Linux

Installer Python 3 et Thonny avec le gestionnaire de paquets de la distribution ou l'installateur officiel. Vérifier dans un terminal :

```
python3 --version
```

Packs de code à utiliser

Dans 09_PACKS_CODE/ :

- 1re_nsi_CODE_ELEVE_COMMUN.zip : fichiers communs sans solution ;
- 1re_nsi_CODE_Ahmad_BELDI.zip : fichiers et consignes du parcours Fondations ;
- 1re_nsi_CODE_Ahmed_BENHADJ_SALEM.zip : fichiers et consignes du parcours Reprise/ Examen ;
- 1re_nsi_CODE_ENSEIGNANT.zip : solutions et tests - à ne jamais copier sur un poste élève.

Arborescence conseillée

```
Premiere_NSI/  
├─ S1_variables_conditions/  
├─ S2_boucles/  
├─ S3_fonctions_tests/  
├─ S4_structures_csv/  
├─ S5_projet_capteurs/  
└─ sauvegardes/
```

Vérification de l'environnement

Créer test_installation.py :

```
from pathlib import Path  
  
print("Python fonctionne")  
print("Dossier courant :", Path.cwd())  
assert 2 + 2 == 4
```

Le programme doit s'exécuter sans erreur.

Règles de sauvegarde

- ne jamais écraser la dernière version qui fonctionne ;
- utiliser des noms comme `projet_v01.py`, `projet_v02.py` ;
- conserver `mesures_capteurs.csv` avec le projet ;
- enregistrer après chaque jalon ;
- ne jamais stocker de mot de passe, clé ou donnée personnelle dans un script ;
- ne pas utiliser de vraies données d'élèves.

En cas de problème

Noter le message exact, le système, la version de Python, le fichier et l'étape qui échoue. Joindre une capture du message complet ; ne pas se contenter de « ça ne marche pas ».



PRÉ-RENTRÉE 2026-2027

Mémento Python

Première NSI - référence de l'élève

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Syntaxe, méthodes, tests et erreurs fréquentes

1. Lire un programme

Python exécute les instructions dans l'ordre, sauf lorsqu'une condition, une boucle ou un appel de fonction modifie le flux.

```
x = 3          # affectation
x = x + 1      # évalue x + 1 puis remplace x
x == 4         # comparaison, renvoie True ou False
```

2. Types et conversions

Type	Exemple	Conversion
entier	12	<code>int("12")</code>
flottant	2.5	<code>float("2.5")</code>
chaîne	"NSI"	<code>str(12)</code>
booléen	True	<code>bool(expression)</code>
absence	None	pas de conversion usuelle

Attention : `input()` renvoie toujours une chaîne.

3. Opérateurs

- calcul : `+` `-` `*` `/` `//` `%` `**` ;
- comparaison : `==` `!=` `<` `<=` `>` `>=` ;
- logique : `and` `or` `not` ;
- appartenance : `in`, `not in`.

4. Conditions

```
if condition:
    ...
elif autre_condition:
    ...
else:
    ...
```

Négations utiles :

Condition	Négation
<code>x > a</code>	<code>x <= a</code>
<code>x >= a</code>	<code>x < a</code>
<code>A and B</code>	<code>(not A) or (not B)</code>
<code>A or B</code>	<code>(not A) and (not B)</code>

5. Boucles

for

```
for i in range(debut, fin, pas):
    ...
```

La valeur `fin` est exclue.

while

```
while condition:
    ...
```

Vérifier que la condition peut devenir fausse.

Schémas

```
compteur = 0
somme = 0
for valeur in valeurs:
    if valeur > 0:
        compteur += 1
    somme += valeur
```

6. Fonctions

```
def moyenne(valeurs: list[float]) -> float:
    """Renvoie la moyenne d'une liste non vide."""
    assert len(valeurs) > 0
    return sum(valeurs) / len(valeurs)
```

- paramètre : nom dans la définition ;
- argument : valeur donnée à l'appel ;
- return : valeur transmise ;
- sans return : résultat None ;
- variable locale : créée dans la fonction.

7. Tests

```
assert est_pair(4) is True
assert est_pair(5) is False
assert maximum_deux(-3, -7) == -3
```

Tester : cas ordinaire, borne, égalité, vide ou invalide selon le contrat.

8. Chaînes, tuples et listes

```
texte = "python"
texte[0]      # 'p'

point = (3, 5)  # tuple

L = [4, 8, 15]
L[0]          # 4
L[-1]         # 15
len(L)        # 3
L.append(16)
```

Compréhension :

```
carres = [x*x for x in L if x >= 0]
```

9. Mutation et copie

```
a = [1, 2]
b = a          # alias
c = a.copy()   # copie superficielle
```

Modifier `b` modifie `a`, mais pas `c`.

10. Dictionnaires

```
mesure = {"id": "C01", "temperature": 28.4}
mesure["temperature"]
mesure["statut"] = "OK"

for cle, valeur in mesure.items():
    print(cle, valeur)
```

11. CSV

```
import csv

with open("mesures_capteurs.csv", encoding="utf-8", newline="") as fichier:
    lignes = list(csv.DictReader(fichier))
```

Les valeurs sont des chaînes : convertir avant de calculer.

12. Algorithmes essentiels

Recherche séquentielle

```
def indice_de(valeurs, cible):
    for i in range(len(valeurs)):
        if valeurs[i] == cible:
            return i
    return None
```

Maximum

```
def maximum(valeurs):
    assert len(valeurs) > 0
    meilleur = valeurs[0]
    for valeur in valeurs[1:]:
        if valeur > meilleur:
            meilleur = valeur
    return meilleur
```

Dichotomie

Nécessite une liste triée. À chaque tour, élimine environ la moitié de la zone restante.

13. Erreurs fréquentes

Erreur	Symptôme	Correction
<code>=</code> au lieu de <code>==</code>	syntaxe ou mauvaise intention	<code>=</code> affecte, <code>==</code> compare
borne de <code>range</code>	un tour en moins	écrire les valeurs produites
index à partir de 1	mauvais élément	premier indice 0
<code>len(L)</code>	confusion avec dernier indice	dernier indice <code>len(L) - 1</code>
oubli de <code>return</code>	résultat <code>None</code>	ajouter <code>return</code> si une valeur est attendue
accumulateur mal initialisé	somme fausse	initialiser avant la boucle
liste vide	erreur au premier élément	définir un contrat
aliasing	modification inattendue	utiliser <code>copy()</code> si nécessaire
CSV non converti	concaténation de chaînes	convertir avec <code>int/float</code>

14. Checklist avant de rendre un programme

- ☐ noms explicites ;
- ☐ fonctions courtes ;
- ☐ docstrings ;
- ☐ aucun code dupliqué ;
- ☐ cas limites traités ;
- ☐ tests exécutés ;
- ☐ sorties lisibles ;
- ☐ aucun affichage de débogage inutile ;
- ☐ programme expliqué oralement.



PRÉ-RENTRÉE 2026-2027

Séance 1 - Cahier élève

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ comprendre l'ordre d'évaluation d'une affectation ;
- ☐ distinguer affectation et comparaison ;
- ☐ identifier les types de base ;
- ☐ écrire une condition et sa négation ;
- ☐ programmer un classificateur robuste ;

Rituel sans ordinateur

Question 1

Tracer sans exécuter :

```
a = 4  
b = a + 2  
a = 10
```

Que valent `a` et `b` à la fin ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Écrire en Python la négation exacte de :

```
age >= 18
```


Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

État et affectation

Une variable est un nom associé à une valeur dans l'état courant du programme. Pour `x = expression`, Python :

1. évalue l'expression à droite ;
2. stocke la valeur obtenue sous le nom à gauche.

Ainsi, après `a = 3` ; `b = a` ; `a = 5`, `b` vaut encore `3`.

Types de base

Type	Exemple	Usage
<code>int</code>	<code>12</code> , <code>-4</code>	entier
<code>float</code>	<code>3.5</code>	approximation d'un réel
<code>str</code>	<code>"NSI"</code>	texte
<code>bool</code>	<code>True</code> , <code>False</code>	condition
<code>NoneType</code>	<code>None</code>	absence de valeur

Booléens et conditions

- `and` exige que les deux conditions soient vraies ;
- `or` exige qu'au moins une condition soit vraie ;
- `not` nie la condition ;
- la négation de `x > 5` est `x <= 5`.

```
if temperature < 0:
    etat = "gel"
elif temperature <= 30:
    etat = "normal"
else:
    etat = "alerte"
```

Activité commune - tables de trace

A. Affectations

Compléter les états successifs.

```
a = 3
b = a
a = 5
c = a + b
```

Ligne exécutée			
départ	non défini	non défini	non défini
a = 3			
b = a			
a = 5			
c = a + b			

B. Conditions aux bornes

Pour chacune des valeurs -1, 0, 5, 10, 11, indiquer la valeur de :

$(x > 0) \text{ and } (x < 10)$

x	résultat	justification
-1		
0		
5		
10		
11		

Parcours Fondations - Ahmad

1. Corriger le programme suivant :

```
age = "16"
print(age + 1)
```

2. Compléter pour afficher `pair`, `impair` ou `nul`.

```
n = -3
# À compléter
```

3. Écrire la négation Python de chacune des conditions :

- `x >= 0` ;
- `age < 18` ;
- `(x > 0) and (x < 10)`.

4. Écrire un programme qui classe une température en `gel`, `normal` ou `alerte`.

Parcours Fiabilisation - Ahmed

1. Écrire une condition vraie exactement lorsque `x` n'appartient pas à l'intervalle `[0, 10]`.
2. Simplifier la négation de `(x >= 2) and (x <= 8)`.
3. Corriger un programme comportant une branche inaccessible.
4. Écrire des tests sur les valeurs limites `-1`, `0`, `30`, `31` du classificateur de température.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 1 - Supports pratiques

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - table mémoire

Découper ou projeter cette grille. Une ligne correspond à l'état **après** l'instruction.

Instruction				Type de 
départ				

Support 2 - cartes de types

int

Entier : 0, 12, -7

Conversion : `int(...)`

float

Nombre approché : 3.5

Conversion : `float(...)`

str

Texte : "NSI"

`input()` renvoie une chaîne.

bool

`True` ou `False`

Produit par une comparaison.

None

Absence de valeur.

Retour implicite d'une fonction sans `return`.

Support 3 - table de vérité

A	B	A and B	A or B	not A
False	False			
False	True			
True	False			
True	True			

Support 4 - cartes de bornes

-1

Juste avant la borne 0

0

Borne inférieure

10

Borne supérieure

11

Juste après la borne 10



PRÉ-RENTRÉE 2026-2027

Séance 1 - Cartes d'aide

Variables, types, booléens et conditions

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Lire

Lis une ligne à la fois et écris la valeur de chaque variable.

B - Type

Demande-toi si la valeur est un entier, un flottant, un texte ou un booléen.

C - Affecter

Évalue d'abord la partie droite, puis remplace la valeur associée au nom de gauche.

D - Condition

Teste séparément chaque comparaison avant de combiner avec and/or.

E - Vérifier

Teste les valeurs limites et trace le programme sans ordinateur.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 2 - Cahier élève

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ comprendre les valeurs produites par range ;
- ☐ choisir entre for et while ;
- ☐ utiliser compteur et accumulateur ;
- ☐ éviter les erreurs de borne ;
- ☐ justifier une terminaison simple ;

Rituel sans ordinateur

Question 1

Écrire, dans l'ordre, les valeurs produites par :

```
range(2, 9, 3)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Tracer puis donner la valeur finale de `s` :

```
s = 0
for i in range(4):
    s = s + i
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Boucle bornée

```
for i in range(3):  
    print(i)
```

produit 0, 1, 2. La borne supérieure est exclue.

Boucle non bornée

```
while condition:  
    # corps
```

Elle convient lorsque le nombre de répétitions n'est pas connu à l'avance. Il faut identifier un **variant** qui évolue vers l'arrêt.

Schémas de base

- compteur : `compteur += 1` ;
- accumulateur : `somme += valeur` ;
- recherche d'extremum : initialiser avec le premier élément, puis comparer ;
- parcours : traiter chaque élément exactement une fois.

Activité commune - prévoir avant d'exécuter

```
s = 0  
for i in range(1, 4):  
    s = s + i
```

Tour	☐	☒ avant	☐ après
1			
2			
3			

Choisir la boucle

Situation	for ou while ?	Justification
parcourir une liste		
demandeur un mot de passe jusqu'à réussite		
afficher 10 nombres		
lire des valeurs jusqu'au mot fin		

Parcours Fondations - Ahmad

1. Écrire les valeurs de `i` pour `range(5)`, `range(2, 6)` et `range(10, 3, -2)`.
2. Compléter un programme qui calcule la somme de `1` à `n`.
3. Compter le nombre de valeurs positives d'une liste.
4. Calculer le maximum sans utiliser `max`.
5. Corriger une boucle `while` qui ne se termine jamais.

Parcours Fiabilisation - Ahmed

1. Écrire une fonction de test manuel pour vérifier une somme cumulée.
2. Calculer simultanément somme, effectif, minimum et maximum en un seul parcours.
3. Traiter proprement la liste vide.
4. Expliquer un invariant du calcul de somme.
5. Écrire une boucle `while` dont le variant est explicite.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 2 - Supports pratiques

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - bandes range

Compléter avant toute exécution.

Expression	Première valeur	Dernière valeur produite	Nombre de tours	Valeurs
range(5)				
range(2, 8)				
range(10, 2, -2)				
range(5, 5)				

Support 2 - cartes de schémas de boucle

Compteur

Compte combien de fois une propriété est vraie.

```
compteur = 0
...
compteur += 1
```

Accumulateur

Construit progressivement une somme ou un résultat.

```
total = 0
...
total += valeur
```


Extremum

Part du premier élément puis compare les suivants.

Recherche

S'arrête ou renvoie un résultat dès que la cible est trouvée.

Support 3 - grille de trace de boucle

Tour	Valeur de boucle	État avant	Instruction	État après	Condition d'arrêt
1					
2					
3					
4					
5					

Support 4 - tri `for` / `while`

Découper puis classer : « parcourir une liste », « attendre un mot de passe valide », « répéter 20 fois », « lire jusqu'au mot fin », « parcourir les caractères d'un texte », « recommencer tant que la saisie est invalide ».



PRÉ-RENTRÉE 2026-2027

Séance 2 - Cartes d'aide

Boucles, compteurs et accumulateurs

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Range

Écris explicitement les valeurs produites par range.

B - Schéma

Cherches-tu à compter, additionner, rechercher ou répéter jusqu'à une condition ?

C - Initialiser

Un compteur et une somme commencent souvent à 0 ; un maximum commence souvent au premier élément.

D - Boucle

Vérifie que la variable de boucle ou la condition évolue.

E - Vérifier

Compte les tours et compare le résultat à un calcul manuel.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 3 - Cahier élève

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ distinguer paramètre et argument ;
- ☐ distinguer print, return et None ;
- ☐ comprendre la portée locale ;
- ☐ documenter une fonction ;
- ☐ écrire des assertions de test ;

Rituel sans ordinateur

Question 1

Que vaut `r` après l'exécution ?

```
def double(x):  
    print(2 * x)  
  
r = double(5)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Dans `def aire_disque(rayon):`, distinguer le paramètre, l'argument dans `aire_disque(3)` et la valeur renvoyée.

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

```
def carre(n: int) -> int:
    """Renvoie le carré de n."""
    return n * n
```

- `n` est un paramètre ;
- dans `carre(5)`, `5` est l'argument ;
- `return` termine la fonction et transmet une valeur ;
- `print` affiche mais ne renvoie pas cette valeur ;
- sans `return`, Python renvoie `None` ;
- les variables créées dans la fonction sont locales.

Contrat

- précondition : ce qui doit être vrai avant l'appel ;
- postcondition : ce qui est garanti après l'appel ;
- `assert` peut vérifier une condition ;
- un jeu de tests ne prouve pas l'absence de bugs, mais il réduit le risque.

Activité commune - prédire les retours

```
def f(x):
    print(x + 1)

resultat = f(4)
```

1. Qu'est-ce qui est affiché ?
2. Que vaut `resultat` ?
3. Comment modifier `f` pour renvoyer 5 ?

Conception de tests

Pour une fonction `maximum_deux(a, b)`, proposer :

- un cas ordinaire ;
- un cas où `a == b` ;
- un cas avec deux valeurs négatives.

Parcours Fondations - Ahmad

1. Écrire `est_pair(n)`.
2. Écrire `maximum_deux(a, b)`.
3. Écrire `compte_occurrences(valeurs, cible)`.
4. Ajouter une docstring à chaque fonction.
5. Écrire deux assertions par fonction.

Parcours Fiabilisation - Ahmed

1. Écrire une précondition pour une fonction de moyenne.
2. Choisir le comportement sur liste vide : assertion ou `None`, et le justifier.
3. Écrire des tests limites et invalides.
4. Corriger une fonction qui modifie une variable globale.
5. Lire la documentation de `math.isclose` et l'utiliser pour tester un flottant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 3 - Supports pratiques

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - anatomie d'une fonction

```
def nom_fonction(parametre_1, parametre_2):  
    """Description brève du résultat et des préconditions."""  
    # calcul local  
    return resultat
```

Élément	Rôle	Exemple personnel
nom	intention	
paramètre	entrée symbolique	
argument	valeur lors de l'appel	
variable locale	état interne	
return	sortie transmise	

Support 2 - cartes « afficher ou renvoyer ? »

print

Affiche pour un humain.

La valeur affichée n'est pas automatiquement réutilisable.

return

Transmet une valeur à l'appelant.

Met fin à l'exécution de la fonction.

None

Valeur obtenue si aucun `return` explicite n'est exécuté.

assert

Vérifie une condition du contrat ou un résultat de test.

Support 3 - fabrique de tests

Fonction	Cas normal	Cas limite	Cas égalité	Cas invalide	Résultat attendu

Support 4 - ticket de débogage

1. Symptôme observé :
2. Plus petit exemple qui échoue :
3. Premier état incorrect :
4. Hypothèse :
5. Test après correction :



PRÉ-RENTRÉE 2026-2027

Séance 3 - Cartes d'aide

Fonctions, contrats, tests et débogage

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Interface

Liste les paramètres et la valeur attendue en sortie.

B - Return

print affiche ; return transmet une valeur à l'appelant.

C - Portée

Une variable créée dans une fonction est locale sauf indication explicite.

D - Contrat

Écris une précondition et une postcondition.

E - Tests

Ajoute un cas normal, un cas limite et un cas invalide.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 4 - Cahier élève

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ maîtriser indexation et longueur ;
- ☐ comprendre mutation et aliasing ;
- ☐ utiliser listes et dictionnaires ;
- ☐ importer une table CSV ;
- ☐ filtrer, trier et contrôler des données ;

Rituel sans ordinateur

Question 1

Pour `L = [4, 8, 15, 16]`, donner `L[1]`, `len(L)` et le dernier indice.

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Prédire la valeur de `a` :

```
a = [1, 2]
b = a
b.append(3)
```

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Listes

- premier élément : `L[0]` ;
- dernier élément : `L[-1]` ;
- longueur : `len(L)` ;
- ajout : `L.append(x)` ;
- parcours : `for valeur in L:` ;
- compréhension : `[x*x for x in L if x >= 0]`.

Alias

```
a = [1, 2]
b = a
b.append(3)
```

`a` et `b` désignent la même liste. Pour une copie superficielle : `b = a.copy()`.

Dictionnaires

```
mesure = {"id": "C01", "temperature": 28.4}
```

Parcours : `for cle, valeur in mesure.items():`.

CSV

La bibliothèque standard `csv` permet d'importer une table. Chaque ligne peut devenir un dictionnaire avec `csv.DictReader`.

Activité commune - index, longueur et mutation

```
L = [4, 8, 15, 16]
```

Expression	Valeur
<code>L[0]</code>	
<code>L[1]</code>	
<code>len(L)</code>	
<code>L[len(L)-1]</code>	

Alias

Prédire puis exécuter :

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Expliquer le résultat.

Parcours Fondations - Ahmad

1. Corriger cinq expressions d'indexation.
2. Écrire une boucle qui double chaque valeur sans modifier la liste d'origine.
3. Construire un dictionnaire décrivant une mesure.
4. Parcourir `keys`, `values` et `items`.
5. Importer `mesures_capteurs.csv` et afficher les identifiants.

Parcours Fiabilisation - Ahmed

1. Démontrer un effet d'aliasing et le corriger.
2. Construire une compréhension filtrant les alertes.
3. Valider chaque ligne du CSV.
4. Détecter un identifiant dupliqué.
5. Trier les mesures par température puis par identifiant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 4 - Supports pratiques

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
Fiches de manipulation, traçage et projet

Support 1 - bande d'indices

Pour $L = [4, 8, 15, 16]$:

Indice	0	1	2	3
Valeur	4	8	15	16

- longueur : 4 ;
- dernier indice : 3 ;
- dernier élément : $L[-1]$.

Support 2 - alias ou copie ?

Alias

```
a = [1, 2]
b = a
```

Deux noms, un seul objet.

Copie superficielle

```
a = [1, 2]
b = a.copy()
```

Deux listes distinctes au premier niveau.

Support 3 - fiche enregistrement

Clé	Valeur	Type attendu	Valide ?
id		str non vide	
date		str ISO	
temperature		float	

Clé	Valeur	Type attendu	Valide ?
humidite		int	
statut		OK ou ALERTE	

Support 4 - mini-table CSV

```
id,date,temperature,humidite,statut
C01,2026-08-24,28.4,55,OK
C02,2026-08-24,31.2,52,ALERTE
C03,2026-08-24,-1.5,80,ALERTE
```

Questions : quelles valeurs doivent être converties ? Quelles contraintes vérifier ? Comment détecter un identifiant dupliqué ?



PRÉ-RENTRÉE 2026-2027

Séance 4 - Cartes d'aide

Listes, tuples, dictionnaires et tables CSV

5 séances de 2 heures - 10 heures
Python 3 - théorie, pratique, tests et projet
À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Indice

Le premier indice est 0 et le dernier $\text{len}(L)-1$.

B - Mutation

Demande-toi si l'opération modifie la liste ou construit une nouvelle liste.

C - Dictionnaire

Accède à une valeur avec sa clé et parcours avec `items()`.

D - CSV

Les valeurs lues sont des chaînes : convertis int/float avant de calculer.

E - Cohérence

Cherche valeurs manquantes, types invalides et identifiants dupliqués.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Séance 5 - Cahier élève

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Mes objectifs

- ☐ écrire une recherche séquentielle ;
- ☐ calculer extremum et moyenne ;
- ☐ comprendre un tri par insertion ;
- ☐ tracer une recherche dichotomique ;
- ☐ livrer un projet documenté et testé ;

Rituel sans ordinateur

Question 1

Dans une liste non triée, quel algorithme simple permet de trouver la première occurrence d'une cible ? Que renvoie-t-on si elle est absente ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Question 2

Quel prérequis est indispensable à la recherche dichotomique et que devient approximativement la zone de recherche à chaque étape ?

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :

Notions de cours

Recherche séquentielle

Parcourt les éléments un à un. Coût proportionnel au nombre d'éléments.

Tri par insertion

Maintient une partie gauche déjà triée et insère l'élément courant à sa place. Le pire cas est quadratique.

Recherche dichotomique

Dans une liste triée, compare à l'élément central et élimine la moitié restante. Le nombre d'étapes croît comme un logarithme.

Correction et terminaison

- invariant : propriété vraie avant et après chaque tour ;
- variant : entier positif qui décroît vers zéro ;
- tests : cas présent, absent, premier, dernier, liste vide.

Activité commune - choisir l'algorithme

Besoin	Algorithme	Précondition	Coût intuitif
trouver un identifiant dans une liste quelconque			
trouver dans une liste triée très longue			
remettre une petite liste dans l'ordre			
trouver la plus grande température			

Projet final

À partir de `mesures_capteurs.csv`, produire un programme qui :

1. charge et valide les données ;
2. calcule moyenne, minimum et maximum ;
3. filtre les alertes ;
4. recherche un capteur par identifiant ;
5. trie les mesures ;
6. affiche un rapport lisible ;
7. contient des fonctions, docstrings et assertions.

Parcours Projet guidé - Ahmad

- utiliser le fichier `projet_final_starter_ahmad.py` ;
- compléter les fonctions dans l'ordre indiqué ;
- exécuter les tests fournis après chaque fonction ;
- expliquer deux fonctions lors de la restitution.

Parcours Projet autonome - Ahmed

- utiliser `projet_final_specification_ahmed.md` ;
- définir l'architecture du programme avant de coder ;
- écrire les tests, y compris liste vide et identifiant absent ;
- comparer recherche séquentielle et dichotomique ;
- expliquer la complexité intuitive et un invariant.

Plan de code ou pseudo-code

Tests prévus avant exécution

Test	Entrée	Résultat attendu	Résultat obtenu	Validé
1				☐
2				☐
3				☐
4				☐

Journal de débogage

Symptôme observé	Hypothèse	Modification testée	Résultat

Trace écrite personnelle

Exit ticket

1. Une construction Python que je sais utiliser :

2. Un test qui m'a aidé :

Ma certitude : ☐ 1 Je devine ☐ 2 J'hésite ☐ 3 Je pense avoir juste ☐ 4 Je peux expliquer

Mon contrôle :



PRÉ-RENTRÉE 2026-2027

Séance 5 - Supports pratiques

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Fiches de manipulation, traçage et projet

Support 1 - cartes d'algorithmes

Recherche séquentielle

Liste quelconque. Examine au pire tous les éléments.

Recherche dichotomique

Liste triée. Élimine environ la moitié de la zone à chaque étape.

Tri par insertion

Maintient une partie gauche triée et insère l'élément courant.

Extremum

Un parcours suffit pour trouver minimum ou maximum.

Support 2 - choix et coût

Besoin	Données triées ?	Algorithme	Nombre d'éléments doublé : effet attendu
trouver une cible	non		
trouver une cible	oui		
trier une petite liste	non		
trouver un maximum	sans importance		

Support 3 - tableau de pilotage du projet

Fonction	Spécification	Écrite	Test normal	Test limite	Validée
charger		☐	☐	☐	☐
valider		☐	☐	☐	☐
statistiques		☐	☐	☐	☐
filtrer		☐	☐	☐	☐
rechercher		☐	☐	☐	☐
trier		☐	☐	☐	☐
rapport		☐	☐	☐	☐

Support 4 - grille d'oral de deux minutes

- problème traité ;
- données d'entrée ;
- décomposition en fonctions ;
- un test important ;
- un bug rencontré ;
- résultat obtenu ;
- amélioration possible.



PRÉ-RENTRÉE 2026-2027

Séance 5 - Cartes d'aide

Algorithmes et mini-projet de synthèse

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

À découper - distribution graduée

Règle d'utilisation

Donner une seule carte à la fois. L'élève inscrit la carte maximale utilisée dans son portfolio.

A - Spécifier

Écris les entrées, sorties et cas limites avant le code.

B - Décomposer

Une fonction par responsabilité : charger, valider, calculer, filtrer, rechercher, afficher.

C - Algorithme

Choisis recherche séquentielle ou dichotomique selon que la liste est triée.

D - Tests

Teste présent, absent, premier, dernier et vide.

E - Expliquer

Prépare une phrase sur la correction, la terminaison et le coût.

Ticket des aides

Exercice	Aucune	A	B	C	D	E	Réussi ensuite
1	☐	☐	☐	☐	☐	☐	☐
2	☐	☐	☐	☐	☐	☐	☐
3	☐	☐	☐	☐	☐	☐	☐
4	☐	☐	☐	☐	☐	☐	☐



PRÉ-RENTRÉE 2026-2027

Mini-diagnostic pratique

Python - 25 minutes

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document élève - sans correction

Partie A - prévoir sans exécuter

1. Affectation

```
a = 2  
b = a + 3  
a = 10
```

Que valent `a` et `b` ?

2. Boucle

```
s = 0  
for i in range(2, 6):  
    s = s + i
```

Compléter la trace et donner `s`.

3. Liste

Pour `L = [5, 8, 13]`, donner `L[1]`, `len(L)` et le dernier indice.

4. Fonction

```
def f(x):  
    print(x * 2)  
  
r = f(3)
```

Qu'est-ce qui est affiché et que vaut `r` ?

Partie B - programmer

Écrire une fonction `compte_supérieurs(valeurs, seuil)` qui renvoie le nombre de valeurs strictement supérieures au seuil.

Contraintes : une boucle, un compteur, un `return`, deux assertions de test.



PRÉ-RENTRÉE 2026-2027

Évaluation finale

Python - théorie et pratique

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Document élève - sans correction

Partie A - connaissances et traçage - 8 points

1. Expliquer la différence entre `=` et `==`.
2. Donner la négation de `(x >= 0)` and `(x < 10)`.
3. Donner les valeurs produites par `range(5, 0, -2)`.
4. Expliquer la différence entre `len(L)` et le dernier indice.
5. Que renvoie une fonction sans `return` ?
6. Pourquoi tester deux flottants avec `==` peut-il être fragile ?
7. Quel prérequis rend possible une recherche dichotomique ?
8. Donner un cas limite pour une fonction calculant une moyenne.

Feuille de réponses - partie A

Q	Réponse
1	
2	
3	
4	
5	
6	
7	

Partie B - pratique - 12 points

Écrire un programme comprenant les fonctions suivantes :

```
def valeurs_valides(valeurs, minimum, maximum):  
    """Renvoie une nouvelle liste contenant les valeurs dans l'intervalle fermé."""  
  
def statistiques(valeurs):  
    """Renvoie un tuple (minimum, maximum, moyenne) pour une liste non vide."""  
  
def indice_capteur(mesures, identifiant):  
    """Renvoie l'indice du premier capteur trouvé, sinon None."""
```

Contraintes :

- ne pas modifier la liste d'origine ;
- ne pas utiliser `min`, `max` ou `sum` dans `statistiques` ;
- écrire au moins six assertions ;
- documenter les préconditions ;
- traiter l'identifiant absent.

Extension

Expliquer comment adapter `indice_capteur` à une recherche dichotomique lorsque les identifiants sont triés.

Fichiers remis

- nom du fichier Python :
- nombre d'assertions exécutées :
- résultat du dernier test :



PRÉ-RENTRÉE 2026-2027

Portfolio individuel

Stage Première NSI - Python

5 séances de 2 heures - 10 heures

Python 3 - théorie, pratique, tests et projet

Nom :

Ma carte de départ

Compétence	0	1	2	3	4	Preuve
Affectation	☐	☐	☐	☐	☐	
Booléens et conditions	☐	☐	☐	☐	☐	
Boucles <code>for</code>	☐	☐	☐	☐	☐	
Boucles <code>while</code>	☐	☐	☐	☐	☐	
Fonctions et <code>return</code>	☐	☐	☐	☐	☐	
Tests	☐	☐	☐	☐	☐	
Listes	☐	☐	☐	☐	☐	
Dictionnaires	☐	☐	☐	☐	☐	
CSV	☐	☐	☐	☐	☐	
Algorithmes	☐	☐	☐	☐	☐	
Autonomie	☐	☐	☐	☐	☐	

Journal des cinq séances

Séance 1

- fichier produit :
- erreur comprise :
- aide maximale : ☐ aucune ☐ A ☐ B ☐ C ☐ D ☐ E
- test dont je suis fier :

Séance 2

- fichier produit :
- schéma utilisé : ☐ compteur ☐ accumulateur ☐ extremum ☐ while
- aide maximale : ☐ aucune ☐ A ☐ B ☐ C ☐ D ☐ E

Séance 3

- fonctions écrites :
- cas limite testé :
- différence `print/return` expliquée : ☐

Séance 4

- structures utilisées : ☐ liste ☐ tuple ☐ dictionnaire ☐ CSV
- erreur d'indexation évitée :

Séance 5

- projet :
- fonction principale expliquée :
- test le plus utile :

Bilan final

Mes trois progrès

1.
2.
3.

Deux priorités pour septembre

1.
2.

Mon engagement

Je programmerai fois par semaine pendant minutes, en conservant un journal de bugs et un jeu de tests.